



Kubernetes-iaC-Deployment — The Lesson

Chain C · Project C-4 · the full written lesson, start to finish

1 — Terraform Provisions the Cluster, Not the Workload

Terraform's job in this project stops at "a working EKS control plane and node group exist." The Deployment, Service, and Ingress that actually run the job-board app are a deliberately separate layer, applied with kubectl/helm — not the kubernetes or helm Terraform providers, which could manage them but would couple a one-line image-tag bump to the same plan/apply cycle as the VPC.

Honest scope note: this machine has no real AWS account or live cluster. Everything here is verified statically — `terraform init -backend=false && terraform validate` succeeds with zero credentials, because validate only checks the configuration's internal consistency, not real cloud state. A real `terraform apply` would still need genuine AWS credentials and could still fail on IAM/quota issues that validate can't see.

2 — Readiness vs. Liveness Probes

Both probes check the same `/health` endpoint, but Kubernetes reacts completely differently to each failing. A failed readinessProbe removes the pod from the Service's endpoints — no traffic reaches it — without touching the container. A failed livenessProbe (repeatedly, past `failureThreshold`) gets the container killed and restarted.

The trap: using livenessProbe to check "has the app finished starting up yet" kills a slow-starting pod in an endless restart loop — it never gets the chance to finish starting, because it keeps getting killed partway through. readinessProbe's `initialDelaySeconds` exists for exactly this: withhold traffic without punishing the pod for still warming up.

3 — Helm: Values In, Manifests Out

`helm template` renders templates against values.yaml (or a `--set` override) into plain Kubernetes YAML. `helm upgrade --set image.tag=<sha>` is how CI deploys a new image without ever editing the chart itself.

The trap: `helm lint` checks the chart's STRUCTURE — it does not render every template combination and verify the output is valid Kubernetes YAML. A typo inside an `{{ if }}` block that only triggers under a specific values override can pass lint cleanly and still produce broken YAML. This project's actual verification pipes `helm template`'s output through `kubeconform`, which is the step that would have caught that.

4 — GitOps with Flux: the Cluster Pulls

The CI pipeline PUSHES a change to the cluster, once, when a pipeline runs. Flux's GitRepository + Kustomization/HelmRelease invert that: the cluster itself PULLS from git on its own interval and reconciles toward whatever it finds — so the repo is the declared desired state, and drift gets corrected rather than accumulating.

The concrete proof: a `kubectl edit` on a CI-push-managed object just sits there until the next deploy happens to overwrite it. Against Flux, the next reconciliation notices the live object doesn't match git and corrects it back — automatically, with no human re-running anything.